LAPORAN PRAKTIKUM

POSTTEST (2)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Alya Kamila Fauziyah (2509106043)

Kelas (A2 ‘25)

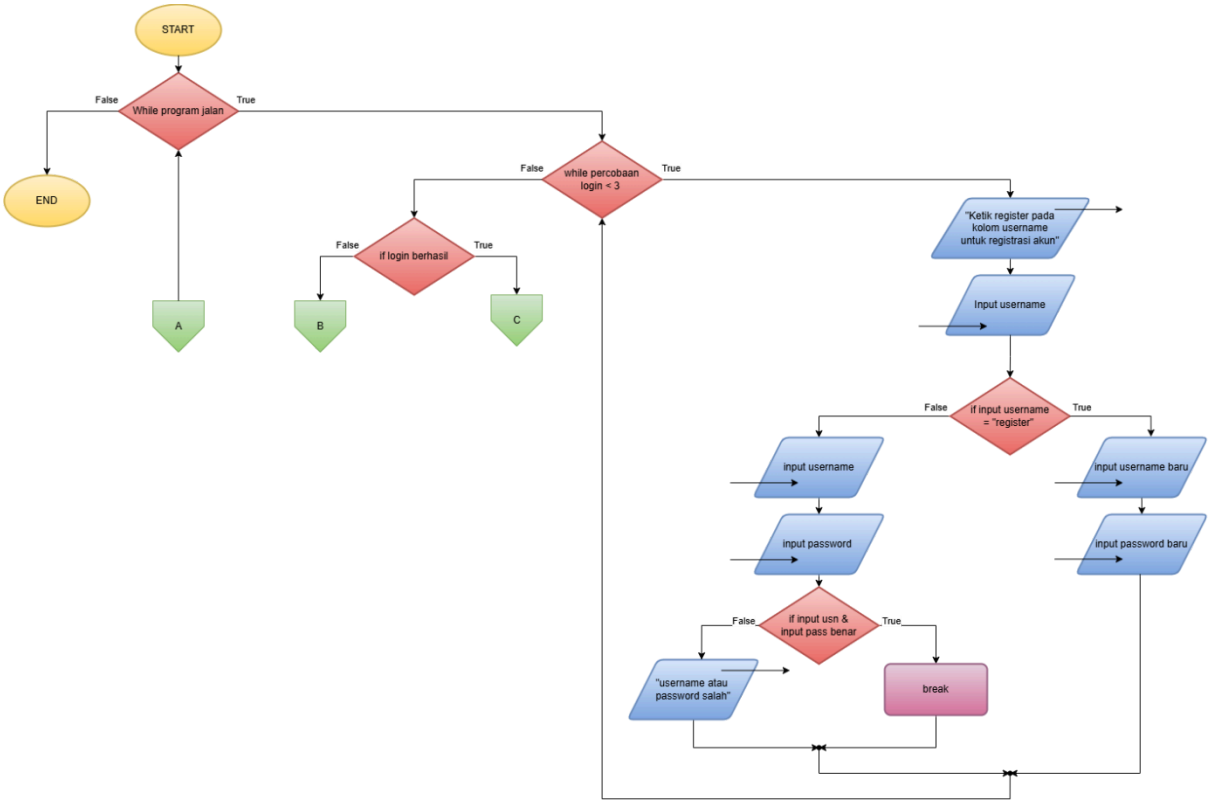
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

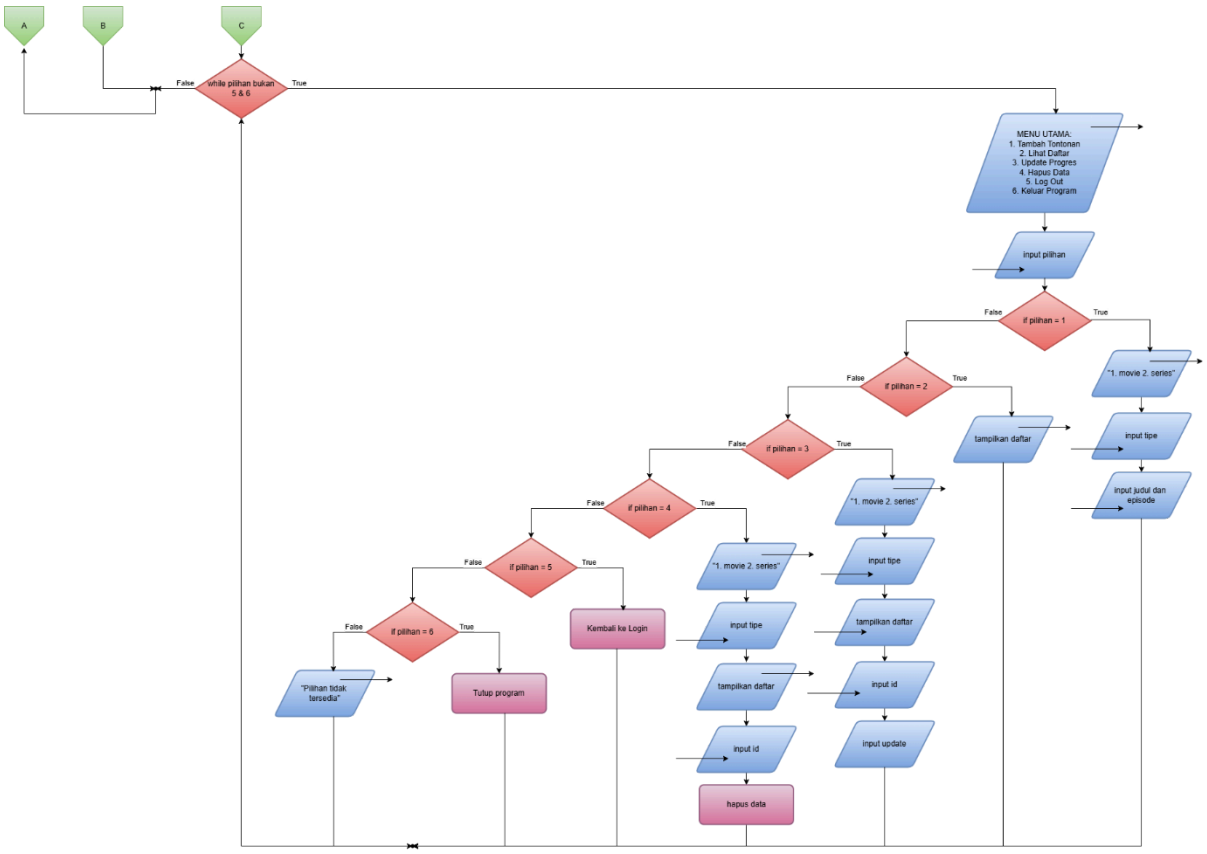
SAMARINDA

2025

1. Flowchart



Gambar 1.1 Flowchart (1)



Gambar 1.2 Flowchart (2)

Program dimulai dengan sistem autentikasi yang berjalan di dalam perulangan utama (while program jalan). Pengguna akan masuk ke sistem keamanan login yang memberikan batas percobaan masuk maksimal sebanyak tiga kali. Pada tahap ini, sistem juga menyediakan percabangan di mana pengguna dapat menginputkan kata "register" pada kolom *username* untuk dialihkan ke proses pembuatan akun baru. Jika pengguna melakukan login, sistem akan memvalidasi *username* dan *password*. Jika data benar, perulangan login akan dihentikan (break), status login berubah menjadi sukses, dan alur program berlanjut menuju konektor C untuk masuk ke menu utama.

Setelah berhasil masuk ke menu utama, program menyediakan antarmuka sistem manajemen tontonan yang dibungkus dalam perulangan while (selama pilihan bukan 5 dan 6). Menu utama ini menyediakan empat fitur pengolahan data utama: Tambah Tontonan, Lihat Daftar, Update Progress, dan Hapus Data. Pada setiap fitur eksekusi data (tambah, update, dan hapus), program menerapkan percabangan bersarang di mana pengguna diwajibkan untuk memilih tipe tontonan terlebih dahulu (Movie atau Series) sebelum

menginputkan detail data seperti judul, jumlah episode, maupun ID target yang ingin diubah atau dihapus.

Perulangan pada menu utama ini memastikan pengguna dapat mengelola daftar tontonannya secara berulang kali tanpa harus kembali ke halaman login. Siklus program di dalam menu utama baru akan benar-benar selesai jika pengguna memasukkan input opsi nomor 5 atau 6. Jika pengguna memilih opsi 5 (Log Out), program akan keluar dari menu utama dan mengembalikan pengguna ke halaman login awal. Namun, jika pengguna memilih opsi 6 (Keluar Program), program akan menghentikan seluruh perulangan utama dan aplikasi akan berakhir secara total.

2. Deskripsi Singkat Program

Program ini adalah aplikasi berbasis konsol yang berfungsi untuk mencatat dan memantau daftar progres tontonan (Movie dan Series). Program dilengkapi dengan sistem keamanan dan registrasi sederhana berupa fitur Login, di mana pengguna wajib memasukkan username dan password yang benar dengan batas maksimal tiga kali percobaan. Jika login gagal hingga batas percobaan tersebut, program akan otomatis berhenti.

Setelah berhasil masuk, pengguna disuguhkan Menu Utama yang memungkinkan mereka memilih opsi untuk menambah, melihat daftar yang terpisah, memperbarui progress, hingga menghapus data tontonan. Program akan terus berjalan menggunakan perulangan hingga pengguna memilih opsi untuk keluar atau logout. Aplikasi ini dirancang untuk memudahkan manajemen daftar hiburan secara cepat, privat, dan interaktif dalam satu sesi penggunaan.

3. Source Code

```
#include <iostream>
#include <string>
#include <limits>

using namespace std;

struct Tontonan {
    int id;
    string judul;
    string tipe;
    int total_eps;
    int eps_ditonton;
    string status;
    string owner;
};

struct User {
    string username;
    string password;
    int movie_nextid;
    int series_nextid;
};

int main() {
    Tontonan daftar[100];
    int jumlah_data = 0;
    User daftar_user[10];
    int jumlah_user = 0;
```

```
    int jumlah_percobaan = 0;
    bool program_berjalan = true;

    while (program_berjalan) {
        string user_in, pass_in;
        bool login_berhasil = false;
        int index_userlogin = -1;

        cout << "\n===================================";
        cout << "\n   SELAMAT DATANG DI MOVIE-TRACK   ";
        cout << "\n===================================";
        cout << "\n(Ketik 'register' untuk buat akun)";
        cout << "\nUsername: "; cin >> user_in;

      if (user_in == "register") {
            if (jumlah_user < 10) {
                                cout << "Buat Username: "; cin >>
daftar_user[jumlah_user].username;
                                cout << "Buat Password: "; cin >>
daftar_user[jumlah_user].password;
                daftar_user[jumlah_user].movie_nextid = 1;
                daftar_user[jumlah_user].series_nextid = 1;
                jumlah_user++;
                cout << "Registrasi Berhasil!\n";
            } else {
                cout << "[Error] Kapasitas user penuh!\n";
            }
            continue;
        } else {
            cout << "Password: "; cin >> pass_in;
```

```
            for (int i = 0; i < jumlah_user; i++) {
                    if (user_in == daftar_user[i].username && pass_in ==
daftar_user[i].password) {
                        login_berhasil = true;
                        index_userlogin = i;
                        break;
                    }
            }

            if (login_berhasil) {
                jumlah_percobaan = 0;
            } else {
                jumlah_percobaan++;
                        cout << "[Error] Username atau Password salah! Sisa
percobaan: " << 3 - jumlah_percobaan << "\n";

                if (jumlah_percobaan >= 3) {
                        cout << "\n[Fatal] Terlalu banyak percobaan gagal.
Program otomatis keluar.\n";
                        program_berjalan = false;
                }
                continue;
            }
        }

        if (login_berhasil) {
            int pilihan;
                                                string current_user =
daftar_user[index_userlogin].username;
```

```
            do {
                  cout << "\n=== MENU UTAMA (User: " << current_user <<
") ===";
                cout << "\n1. Tambah Tontonan";
                cout << "\n2. Lihat Daftar (Terpisah)";
                cout << "\n3. Update Progress";
                cout << "\n4. Hapus Data";
                cout << "\n5. LOGOUT";
                cout << "\n6. KELUAR PROGRAM";
                cout << "\nPilih menu: "; cin >> pilihan;

                if (cin.fail()) {
                    cin.clear();
                          cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                    cout << "\n[Error] Input harus berupa angka!\n";
                    continue;
                }

                if (pilihan == 1) {
                    if (jumlah_data >= 100) {
                              cout << "\n[Pemberitahuan] Maaf, kapasitas
daftar tontonan sudah penuh (100/100)!\n";
                          cout << "Silakan hapus beberapa data terlebih
dahulu untuk menambah judul baru.\n";
                        continue;
                    }

                    cout << "\nTipe (1. Movie, 2. Series, 0. Batal): ";
                    int pt; cin >> pt;
```

```
                    if (cin.fail() || pt < 0 || pt > 2) {
                        cin.clear();
                           cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                             cout << "[Error] Pilihan tipe tidak valid.
Dibatalkan.\n";
                        continue;
                    }
                    if (pt == 0) continue;


                         cout << "Judul: "; cin.ignore(); getline(cin,
daftar[jumlah_data].judul);
                    if (pt == 1) {
                        daftar[jumlah_data].tipe = "Movie";
                                                daftar[jumlah_data].id  =
daftar_user[index_userlogin].movie_nextid++;
                        daftar[jumlah_data].total_eps = 1;
                    } else {
                        daftar[jumlah_data].tipe = "Series";
                                                daftar[jumlah_data].id  =
daftar_user[index_userlogin].series_nextid++;
                                    cout << "Total Episode: "; cin >>
daftar[jumlah_data].total_eps;


                         if (cin.fail() || daftar[jumlah_data].total_eps
<= 0) {
                            cin.clear();

cin.ignore(numeric_limits<streamsize>::max(), '\n');
                                    cout << "[Error] Jumlah episode tidak
valid. Dibatalkan.\n";

daftar_user[index_userlogin].series_nextid--;
                            continue;
```

```
                        }
                    }
                    daftar[jumlah_data].eps_ditonton = 0;
                    daftar[jumlah_data].status = "Rencana Menonton";
                    daftar[jumlah_data].owner = current_user;
                    jumlah_data++;
                    cout << "Data berhasil disimpan!\n";
                }
                else if (pilihan == 2) {
                    cout << "\n>> DAFTAR MOVIE <<";
                    cout << "\nID\tJudul\t\t\tStatus\n";
                    bool ada_movie = false;
                    for (int i = 0; i < jumlah_data; i++) {
                                if (daftar[i].owner == current_user &&
daftar[i].tipe == "Movie") {
                                    cout << daftar[i].id << "\t" <<
daftar[i].judul << "\t\t\t" << daftar[i].status << endl;
                            ada_movie = true;
                        }
                    }
                    if(!ada_movie) cout << "(Kosong)\n";


                    cout << "\n>> DAFTAR SERIES <<";
                    cout << "\nID\tJudul\t\t\tProgress\tStatus\n";
                    bool ada_series = false;
                    for (int i = 0; i < jumlah_data; i++) {
                                if (daftar[i].owner == current_user &&
daftar[i].tipe == "Series") {
                                    cout << daftar[i].id << "\t" <<
daftar[i].judul << "\t\t\t"
                                << daftar[i].eps_ditonton << "/" <<
daftar[i].total_eps << "\t\t" << daftar[i].status << endl;
```

```
                            ada_series = true;
                        }
                    }
                    if(!ada_series) cout << "(Kosong)\n";
                }
                else if (pilihan == 3) {
                        cout << "\nUpdate Apa? (1. Movie, 2. Series, 0.
Batal): ";
                    int up; cin >> up;

                    if (cin.fail() || up < 0 || up > 2) {
                        cin.clear();
                           cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                                cout << "[Error] Pilihan tidak valid.
Dibatalkan.\n";
                        continue;
                    }
                    if (up == 0) continue;

                    string tipe_cari;
                    if (up == 1) {
                        tipe_cari = "Movie";
                    } else {
                        tipe_cari = "Series";
                    }

                         cout << "\n--- Daftar " << tipe_cari << " Anda
---";
                    cout << "\nID\tJudul\n";
                    for (int i = 0; i < jumlah_data; i++) {
```

```
                                if (daftar[i].owner == current_user &&
daftar[i].tipe == tipe_cari) {
                                    cout << daftar[i].id << "\t" <<
daftar[i].judul << endl;
                        }
                    }


                    cout << "\nMasukkan ID (0 Batal): "; int idC; cin
>> idC;
                    if (cin.fail()) {
                        cin.clear();
                            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                        cout << "[Error] ID harus angka!\n";
                        continue;
                    }
                    if (idC == 0) continue;


                    bool ketemu = false;
                    for (int i = 0; i < jumlah_data; i++) {
                            if (daftar[i].id == idC && daftar[i].owner ==
current_user && daftar[i].tipe == tipe_cari) {
                                ketemu = true;
                                if (tipe_cari == "Movie") {
                                        daftar[i].eps_ditonton = 1;
daftar[i].status = "Selesai";
                                    cout << "Berhasil diupdate menjadi
Selesai!\n";
                                } else {
                                    cout << "Episode terakhir ditonton: ";
int eb; cin >> eb;
                                    if (cin.fail() || eb < 0 || eb >
daftar[i].total_eps) {
```

```
                                    cin.clear();

cin.ignore(numeric_limits<streamsize>::max(), '\n');
                                        cout << "[Error] Input episode
tidak valid. Dibatalkan.\n";
                                } else {
                                    daftar[i].eps_ditonton = eb;

                                    if (eb == daftar[i].total_eps) {
                                        daftar[i].status = "Selesai";
                                    } else {
                                            daftar[i].status = "Sedang
Ditonton";
                                    }

                                    cout << "Berhasil diupdate!\n";
                                }
                            }
                            break;
                        }
                    }
                            if (!ketemu) cout << "[Error] ID tidak
ditemukan.\n";
                }
                else if (pilihan == 4) {
                    cout << "\nTipe yang dihapus (1. Movie, 2. Series,
0. Batal): "; int th; cin >> th;
                    if (cin.fail() || th < 0 || th > 2) {
                        cin.clear();
                            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                        cout << "[Error] Pilihan tidak valid.\n";
```

```
                        continue;
                    }
                    if (th == 0) continue;

                    string tipe_hapus;
                    if (th == 1) {
                        tipe_hapus = "Movie";
                    } else {
                        tipe_hapus = "Series";
                    }

                        cout << "\n--- Daftar " << tipe_hapus << " Anda
---";
                    cout << "\nID\tJudul\n";
                    bool ada_data = false;
                    for (int i = 0; i < jumlah_data; i++) {
                                if (daftar[i].owner == current_user &&
daftar[i].tipe == tipe_hapus) {
                                    cout << daftar[i].id << "\t" <<
daftar[i].judul << endl;
                            ada_data = true;
                        }
                    }

                    if (!ada_data) {
                        cout << "(Tidak ada data untuk dihapus)\n";
                        continue;
                    }

                          cout << "\nMasukkan ID yang ingin dihapus (0
Batal): "; int idH; cin >> idH;
```

```
                    if (cin.fail()) {
                        cin.clear();
                           cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                        cout << "[Error] ID harus angka!\n";
                        continue;
                    }
                    if (idH == 0) continue;

                    bool ketemu = false;
                    int indeks_hapus = -1;

                    for (int i = 0; i < jumlah_data; i++) {
                           if (daftar[i].id == idH && daftar[i].owner ==
current_user && daftar[i].tipe == tipe_hapus) {
                            ketemu = true;
                            indeks_hapus = i;
                            break;
                        }
                    }

                    if (ketemu) {
                        char yakin;
                                 cout << "Yakin ingin menghapus '" <<
daftar[indeks_hapus].judul << "'? (y/n): ";
                        cin >> yakin;

                        if (yakin == 'y' || yakin == 'Y') {
                              for (int j = indeks_hapus; j < jumlah_data
- 1; j++) {
                                daftar[j] = daftar[j + 1];
```

```
                            }
                            jumlah_data--;
                            cout << "Data berhasil dihapus!\n";
                        } else {
                            cout << "Penghapusan dibatalkan.\n";
                        }
                    } else {
                        cout << "[Error] ID tidak ditemukan.\n";
                    }
                }
                else if (pilihan == 5) {
                     char yakin; cout << "Yakin Logout? (y/n): "; cin >>
yakin;
                    if (yakin != 'y' && yakin != 'Y') pilihan = 0;
                }
                else if (pilihan == 6) {
                       char yakin; cout << "Yakin Keluar Program? (y/n):
"; cin >> yakin;
                      if (yakin == 'y' || yakin == 'Y') program_berjalan
= false;
                    else pilihan = 0;
                }
                else {
                    cout << "[Error] Pilihan menu tidak tersedia!\n";
                }
            } while (pilihan != 5 && pilihan != 6);
        }
    }
    return 0;
}
```

4. Hasil Output

```
=====================================
   SELAMAT DATANG DI MOVIE-TRACK
=====================================
(Ketik 'register' untuk buat akun)
Username: register
Buat Username: yar
Buat Password: 043
Registrasi Berhasil!
```

Gambar 4.1 Register

```
=====================================
   SELAMAT DATANG DI MOVIE-TRACK
=====================================
(Ketik 'register' untuk buat akun)
Username: yar
Password: 043
```

Gambar 4.2 Log In

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu:
```

Gambar 4.3 Menu Utama

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 1

Tipe (1. Movie, 2. Series, 0. Batal): 1
Judul: Even if This Love Disappear from the World Tonight
Data berhasil disimpan!
```

Gambar 4.4 Menu 1

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 2

>> DAFTAR MOVIE <<
ID      Judul                   Status
1       Even if This Love Disappear from the World Tonight                    Rencana Menonton
2       Interstellar                    Rencana Menonton
3       Call Me by Your Name                    Rencana Menonton

>> DAFTAR SERIES <<
ID      Judul                   Progress        Status
1       Evangelion                      0/26            Rencana Menonton
2       Sonny Boy                       0/12            Rencana Menonton
3       Madoka Magica                   0/12            Rencana Menonton
```

Gambar 4.5 Menu 2

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 3

Update Apa? (1. Movie, 2. Series, 0. Batal): 1

--- Daftar Movie Anda ---
ID      Judul
1       Even if This Love Disappear from the World Tonight
2       Interstellar
3       Call Me by Your Name

Masukkan ID (0 Batal): 3
Berhasil diupdate menjadi Selesai!
```

Gambar 4.6 Menu 3

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 4

Tipe yang dihapus (1. Movie, 2. Series, 0. Batal): 2

--- Daftar Series Anda ---
ID      Judul
1       Evangelion
2       Sonny Boy
3       Madoka Magica

Masukkan ID yang ingin dihapus (0 Batal): 2
Yakin ingin menghapus 'Sonny Boy'? (y/n): y
Data berhasil dihapus!
```

Gambar 4.7 Menu 4

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 5
Yakin Logout? (y/n): y

====================================
   SELAMAT DATANG DI MOVIE-TRACK
====================================
(Ketik 'register' untuk buat akun)
Username:
```

Gambar 4.8 Menu 5

```
=== MENU UTAMA (User: yar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. LOGOUT
6. KELUAR PROGRAM
Pilih menu: 6
Yakin Keluar Program? (y/n): y
PS C:\Praktikum-APL\Post-test>
```

Gambar 4.9 Menu 6

5. Langkah-langkah GIT

5.1 GIT Add

```
PS C:\Praktikum-APL\Post-test> git add .
```

Gambar 5.1 GIT Add

Perintah git add . digunakan untuk memasukkan semua perubahan file di direktori saat ini ke dalam *staging area*. Proses ini merupakan tahap persiapan sebelum perubahan tersebut direkam secara permanen ke dalam *repository* menggunakan perintah *commit*.

5.2 GIT Commit

```
PS C:\Praktikum-APL\Post-test> git commit -m "posttest 2"
[main c8cccb6] posttest 2
 2 files changed, 319 insertions(+)
 create mode 100644 Post-test-APL-2/2509106043-AlyaKamilaFauziyah-PT-2.cpp
 create mode 100644 Post-test-APL-2/2509106043-AlyaKamilaFauziyah-PT-2.exe
```

Gambar 5.2 GIT Commit

Perintah git commit -m berfungsi untuk menyimpan semua perubahan dari *staging area* secara permanen ke dalam *repository* lokal. Opsi -m digunakan untuk menyertakan pesan deskriptif singkat secara langsung yang menjelaskan perubahan apa saja yang telah dilakukan pada versi tersebut.

5.3 GIT Push

```
PS C:\Praktikum-APL\Post-test> git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 81.08 KiB | 10.13 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/yaralyar07/praktikum-apl.git
   ab37235..c8cccb6  main -> main
PS C:\Praktikum-APL\Post-test>
```

Gambar 5.3 GIT Push

Perintah git push berfungsi untuk mengunggah semua perubahan yang sudah di-*commit* dari *repository* lokal ke *repository* jarak jauh (*remote*), seperti GitHub. Langkah ini memastikan versi terbaru dari kodemu tersimpan secara *online* sehingga aman dan dapat diakses dari perangkat lain.